Microsoft Purview Health Check

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Purview Health Check 5

The problem . . . 5

The solution . . . 5

How it works . . . 5

Why it matters . . . 5

Important boundaries . . . 6

Solution Overview — Microsoft Purview Health Check 6

What this solution delivers . . . 6

The three health domains (the rubric) . . . 7

How it works (the assessment model) . . . 7

Important boundaries — read this first . . . 7

What's in the box . . . 8

Requirements — Microsoft Purview Health Check 8

R1. Read access to the Purview compliance surfaces you want to assess . . . 9

R2. Security & Compliance PowerShell + Exchange Online + Graph toolchain . . . 9

R3. A defined in-scope workload + data map (what "covered" should mean) . . . 9

R4. The intended Purview capability baseline (what *should* exist) . . . 10

R5. Role-review boundaries and privacy posture for sensitive surfaces . . . 10

R6. A named owner for each domain (so findings become actions) . . . 10

R7. Change control for the single safe direct fix (audit-log re-enable) and for downstream remediation . . . 11

R8. Compliance, records, privacy & legal stakeholders (determinations stay with them) . . . 11

Checklist . . . 11

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 12

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 13

5. Compliance, records, privacy & legal stakeholders . . . 13

6. A pilot scope and change control . . . 13

Implementation — Microsoft Purview Health Check 14

Phase 0 — Assess (run the health check, read-only) . . . 14

Phase 1 — Design (score the rubric, build the register) . . . 15

Phase 2 — Produce the scorecard (leadership rollup) . . . 16

Phase 3 — Remediate (plan first; commit only the one safe fix) . . . 16

Phase 4 — Validate (re-score, read-only) . . . 17

Phase 5 — Expand (make it recurring) . . . 17
Testing & Validation — Microsoft Purview Health Check 18
Gate 1 — Read access and toolchain are in place . . . 18
Gate 2 — The inventory is complete (Discovery) . . . 18
Gate 3 — Policy health (P) is scored against intent . . . 18
Gate 4 — The unified audit log status is correctly determined . . . 18
Gate 5 — Coverage (C) is scored against the in-scope map . . . 19
Gate 6 — Roles (R) least-privilege review is clean or flagged . . . 19
Gate 7 — Insider-risk / sensitive surfaces reported as POSTURE only . . . 19
Gate 8 — The scorecard and register are complete and owned . . . 19
Gate 9 — Remediation planned `-WhatIf`; only the audit-log fix committed . . . 19
Gate 10 — Re-score confirms improvement (post-remediation) . . . 19
Troubleshooting — Microsoft Purview Health Check 20
A surface returns access-denied → a false "Gap" . . . 20
A cmdlet is not available in the tenant . . . 20
The unified audit-log status is ambiguous or unreadable . . . 21
Coverage is mis-mapped (a policy looks like it covers less/more than it does) . . . 21
Role read is incomplete (Global Admin count missing) . . . 21
Licensing gap scored as a drift Gap (confusion) . . . 22
"Stuck in test" flagged on a fresh pilot (false At Risk) . . . 22
Insider-risk / sensitive surface — how to stay on the right side of the privacy line . . . 22
The scorecard diverges from the portal . . . 22
A determination item was actioned before governance signed off . . . 22
Best Practices — Microsoft Purview Health Check 23
1. Score against intent, not against a maximum . . . 23
2. Read-only first — the check itself changes nothing . . . 23
3. "Stuck in test" is drift, not safety . . . 23
4. Coverage is where the real risk hides . . . 24
5. Treat the unified audit log as foundational . . . 24
6. Least privilege is a health metric, not a one-time setup . . . 24
7. Respect privacy on the sensitive surfaces . . . 24
8. Severity + owner is what turns findings into fixes . . . 24
9. Defer determinations — you provide the method, not the verdict . . . 25
10. Make it recurring — drift comes back . . . 25
11. Verify against your tenant — the surfaces evolve . . . 25
Deliverables Summary — Microsoft Purview Health Check 25

Documentation 25
PowerShell tools (5) 26
Named artifacts (verbatim — used across docs, scripts, and tests) 27
What this solution deliberately does NOT do 28
Appendix A — PowerShell Scripts 29

Executive Summary — Microsoft Purview Health Check

The problem

Microsoft Purview is not "set it and forget it." Over time, DLP policies get paused for an incident and never re-enabled; sensitivity labels stop at the pilot group and never widen; a retention policy covers email but nobody added SharePoint; the unified audit log is quietly off; role groups fill with people who changed teams two reorganizations ago. None of this throws an error — it just **drifts**. The gap becomes visible only during an audit, a data-subject request, or a breach — the moment it is most expensive and least forgiving to discover.

The solution

This solution is a **read-only health check** of your Microsoft Purview estate across three domains:

- **Policy health** — do the policies that should exist actually exist, and are they **enabled and healthy — or** disabled**,** unhealthy**, or** stuck in test **long after the pilot ended (including whether the** unified audit log** is on)?
- **Coverage** — do those policies actually reach the **workloads in scope** (Exchange, SharePoint, OneDrive, Teams, endpoints) — or does "we have DLP" quietly mean "on email only"?
- **Roles** — are the compliance role groups administered at **least privilege**, with no **Global Admin sprawl**, no** single-point-of-failure**, and no** stale** members?

It produces a **health scorecard** (each check rated **Healthy** → **At Risk** → **Gap**), a **prioritized remediation register** (every gap with a **severity** and a named **owner**), and a **remediation roadmap** — without changing anything in the tenant.

How it works

We follow a repeatable model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — adapted for a read-first assessment. We **read** the estate (nothing changes), **score** it against what the organization *intended* to run, and **prioritize** the gaps by severity and owner. Fixes are then routed to the capability solution that owns them (DLP, labels, retention, roles), where they are **piloted in simulation/test/report-only** before enforcement. The **only** fix this solution applies directly is the single safe, reversible one — **re-enabling the unified audit log** — and even that is offered as a preview first and committed only on an explicit switch.

The solution ships as **self-guided documentation plus five PowerShell tools** (Security & Compliance PowerShell / Exchange Online / Graph): an estate **inventory**, a **coverage + least-privilege role** review, a **validation** checker, a **remediation-plan** generator, and a **scorecard** rollup — all **read-only or** `-WhatIf` **by default**, with **no credentials in code**.

Why it matters

- **Catch drift before an audit does** — find the disabled policy, the coverage blind spot, and the role sprawl on your schedule, not a regulator's.

• **Evidence, not opinion** — a scored, defensible picture of estate health and a register you can actually work down.

• **Prioritized and owned** — every gap has a severity and an owner, so the highest-risk items get fixed first instead of everything staying red.

• **Safe by design** — read-only, with high-impact fixes piloted in their owning solutions; the health check finds, it does not flip.

• **A fraction of consulting cost** — the same disciplined method a specialist would use, delivered as a repeatable, re-runnable check you can schedule.

Important boundaries

The health check reports **technical** state and evidence — it does **not** make **records, retention-schedule, legal-hold, or regulatory determinations**; those are **compliance / records / privacy / legal** decisions, and such findings are flagged **"determination required"** and routed accordingly. On **Insider Risk** and **Communication Compliance** it reports **posture only** (that the capability exists and its privacy guardrails are on) — never case contents or any person's identity. Purview is **licensing-dependent** (many advanced capabilities need Microsoft 365 E5 / E5 Compliance; some are in preview) and the portal is **evolving** — a capability that is absent because it is **unlicensed** is a licensing decision, not a drift gap, and capability availability and paths should be **verified against your licensing and tenant**.

Solution Overview — Microsoft Purview Health Check

What this solution delivers

Microsoft Purview is not a "set it and forget it" platform. DLP policies get paused for an incident and never re-enabled; sensitivity labels stop at the pilot group and never widen; a retention policy covers Exchange but nobody added SharePoint; the unified audit log is silently off; role groups accumulate members who changed teams two reorganizations ago. None of this throws an error — it just **drifts**, and the gap only becomes visible during an audit, a data-subject request, or a breach, when it is far too late and far more expensive to fix.

This solution answers one question with **evidence**: **"Is our Microsoft Purview estate actually healthy — configured to policy, covering what it should, and administered by the right people at least privilege — or has it drifted?"** It is a **read-only health check** across three domains — **Policy health, Coverage, and Roles** — that produces a **health scorecard**, a **prioritized remediation register**, and a **remediation roadmap**, without changing anything in the tenant.

> ***This is an *assessment*, not an implementation.*** *The health check **finds and prioritizes** the gaps; **closing** most of them is done through the matching capability solution (DLP, sensitivity labels, retention/records, insider risk, audit) under that solution's own pilot-in-simulation, validate, expand discipline. The one exception this solution will remediate directly is the single clearly-safe, reversible fix — **re-enabling the unified audit log** — and even that is offered `-WhatIf` first and only committed with an explicit switch. Everything else is a **planned** remediation routed to its owning capability and its stakeholders.*

The three health domains (the rubric)

Each check is scored **Healthy** → **At Risk** → **Gap** against named, evidenced criteria, and rolls up into a per-domain and overall **health rating**.

1. **Policy health (P).** Do the Purview policies that should exist **exist, are they enabled, and are

they healthy **— or are they** disabled**, in** error/unhealthy **state, or** stuck in simulation/test **long after the pilot should have ended? This covers** DLP **policies (mode + health),** sensitivity labels and label policies **(published, not orphaned),** retention / data-lifecycle **policies and labels, the** unified audit log **(on/off), and whether an** insider-risk** signal exists where the organization intends one.

1. **Coverage (C).** Do the policies that exist actually **reach the workloads and data in scope** —

Exchange, SharePoint, OneDrive, Teams, and (for endpoint actions) **onboarded devices** — or are there **blind spots** where an in-scope workload has **no** DLP / retention / label coverage? This is where "we have DLP" quietly means "we have DLP on email only".

1. **Roles (R).** Are the Purview/compliance role groups administered at **least privilege** — with

no Global Administrator sprawl, **no single-point-of-failure** role group (one member, no coverage), **no empty** role groups, and **no stale** members who no longer need access? Purview uses fine-grained role groups; drift here is a real, auditable risk.

How it works (the assessment model)

The health check follows the standard Purview model, adapted for a read-first assessment:

1. **Assess (read-only).** Inventory the current state — DLP policies and their **mode**, sensitivity

labels and label policies, retention/lifecycle policies and labels, unified-audit-log status, insider-risk policy presence, and compliance role-group membership — with the included **read-only** scripts. Nothing changes.

1. **Design (the scorecard).** Score each check against the rubric (**Healthy / At Risk / Gap**),

record every gap in the **Remediation Register** with **severity** (High / Medium / Low) and an **owner**, and produce the overall **health rating** and remediation order.

1. **Remediate (plan first, simulation/test where it changes anything).** Generate the prioritized

remediation plan (`-WhatIf` by default). Route each remediation to its **owning capability solution** (DLP, labels, retention, roles), where it is piloted in **simulation/test/report-only** before enforcement. The **only** fix this solution commits directly is **re-enabling the unified audit log** — offered `-WhatIf` first, committed only with an explicit switch, and trivially reversible.

1. **Validate.** Re-run the read-only validation gates to confirm each remediated item now reads

Healthy, and re-score.

1. **Expand (operationalize).** Turn the health check into a **recurring** cadence so drift is caught

early next time — the scripts are re-runnable read-only and produce a comparable scorecard each run.

Important boundaries — read this first

- **Records, retention schedules, and legal/regulatory determinations are not ours to make.** The

health check reports **technical** state (a retention policy exists / covers these workloads / is in this mode) and evidence. **Whether** a given retention schedule, records file plan, legal hold, or regulatory obligation is correct or complete is a **records / compliance / privacy / legal determination** — deferred to those functions (`global-requirements-purview.md` §5).

- **Insider Risk & Communication Compliance are HR/privacy-sensitive.** Where the health check reports

on insider-risk configuration, it reports **presence and privacy posture** (e.g. anonymization on, role separation intact) only — never case contents, never a judgement about any person. Reviewing those surfaces respects **role separation** and **privacy-by-design**; outcomes are **HR/legal's**.

- **Remediation of DLP, labels, retention, and insider-risk is high-impact — pilot first.** This

solution deliberately does **not** flip those into enforcement. It hands each gap to the owning capability solution, which pilots in **simulation/test/report-only**, validates, and only then enforces.

- **Purview is licensing-dependent and evolving.** Many checks (advanced DLP, auto-labeling, insider

risk, Audit Premium, DSPM) require **Microsoft 365 E5 / E5 Compliance**; some capabilities are in preview and portal paths change. A capability that is **absent because it is unlicensed** is a **licensing** finding, not a drift finding — the health check flags the distinction and says **"verify against your licensing/tenant."**

What's in the box

- This overview, a **Requirements** guide, an ordered **Implementation** guide (run the health check

→ score → build the register → remediate/validate), **Testing** (pass/fail gates), **Troubleshooting**, **Best Practices**, and a **Deliverables Summary** + **Executive Summary**.

- **Five PowerShell tools** (Security & Compliance PowerShell / Exchange Online / Graph): a **Discovery**

inventory, a **Scope/health** coverage-and-role review, a **Validation** checker, a **Remediation** plan generator (`-WhatIf` first; commits only the safe audit-log re-enable), and a **Reporting** scorecard rollup — all **read-only or** `-WhatIf` **by default**, with **no credentials in code**.

> ***Sources:*** *Microsoft Learn — *Data Loss Prevention* (policy modes/health, simulation), *Sensitivity labels & label policies*, *Data lifecycle management & records* (retention policies/labels), *Microsoft Purview Audit* (unified audit log status), *Insider Risk Management* (privacy/role separation), and *Permissions in the Microsoft Purview portal* (role groups). Purview is* ***licensing-dependent*** *(many advanced capabilities need Microsoft 365 E5 / E5 Compliance; some are in preview) and the portal is* ***evolving*** *—* ***verify capability availability and portal paths against your licensing/tenant*** *rather than assuming.*

Requirements — Microsoft Purview Health Check

Confirm every prerequisite below **before** you run the health check. Category-level prerequisites (tenant, licensing model, least-privilege roles, S&C PowerShell / Graph toolchain, in-scope data sources, compliance/privacy/legal stakeholders, pilot scope + change control) are in `global-requirements-purview.md` — reference them; they are **not** repeated here. The items below are the ones **specific to running a read-only Purview health check** and turning its findings into a

prioritized, owned remediation register.

Each requirement is written as **Description · Why · Verify · Implement if missing.**

> *This is a **read-only assessment**. The only requirement that grants write access is R7, and it is needed **only** if you intend to use the single safe direct fix (re-enabling the unified audit log) — everything else runs read-only. Reviewing insider-risk and role surfaces is **privacy- and HR-sensitive**: read the boundaries in R5 and R8 before you look.*

R1. Read access to the Purview compliance surfaces you want to assess

Description: An account with **read** visibility across the Purview surfaces in scope for the check — **DLP**, **sensitivity labels / label policies**, **retention / data-lifecycle**, **audit configuration**, and **role groups**. Least-privilege read roles (e.g. **Compliance Administrator** or **Compliance Data Administrator** for policy read, **Global Reader** for broad read, and the relevant **View-Only** role groups) are sufficient for the assessment itself. **Why:** The health check reads policy, label, retention, audit, and role state; it needs read access to each surface it scores, but it does **not** need write access to assess. **Verify:** In the Purview portal, confirm you can open **Data Loss Prevention** → **Policies**, **Information Protection** → **Labels / Label policies**, **Data Lifecycle Management** → **Retention**, and **Roles & scopes / Permissions**. `OPSHEALTH-Discovery.ps1` reports what it can and cannot read. **Implement if missing:** Assign a least-privilege **read** role for the surfaces you intend to score, under change control. Do **not** over-grant write access "to be safe" — the assessment is read-only.

R2. Security & Compliance PowerShell + Exchange Online + Graph toolchain

Description: The **ExchangeOnlineManagement** module (provides **Security & Compliance PowerShell** `Connect-IPPSSession` **and** Exchange Online `Connect-ExchangeOnline`, used for the unified-audit-log status check) and, for role/CA/Graph-side signals, the **Microsoft.Graph** modules (category GR §3), with a modern-auth / MFA-capable admin. **Why:** The five scripts run through these — DLP/label/retention/role objects via S&C PowerShell, unified-audit-log status via Exchange Online, and optional directory-role signals via Graph. Cmdlet and Graph parameters are version-dependent — **verify in your tenant. Verify:** `Get-Module -ListAvailable ExchangeOnlineManagement, Microsoft.Graph`; `Connect-IPPSSession`, `Connect-ExchangeOnline`, and `Connect-MgGraph` succeed with modern auth (MFA). **Implement if missing:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser` and `Install-Module Microsoft.Graph -Scope CurrentUser`. **Modern auth only — no stored credentials or app secrets in scripts.**

R3. A defined in-scope workload + data map (what "covered" should mean)

Description: A short, agreed list of the **workloads and data sources** Purview is *supposed* to cover — Exchange, SharePoint, OneDrive, Teams, endpoints/devices — and, where relevant, the **classification/sensitivity intent** (which labels/SITs matter). This is the **yardstick** the Coverage (C) domain scores against. **Why:** "Coverage" is only meaningful relative to an intended scope. Without a target, the check can report *what is covered* but cannot tell you *what is missing*. (Extends category GR §4.) **Verify:** A scope note lists the in-scope workloads and any deliberate out-of-scope items; `OPSHEALTH-ScopeCheck.ps1` reports actual policy coverage per workload to compare against it.

Implement if missing: Capture the intended scope with stakeholders (one page is enough) before scoring Coverage; absence of a target is itself an early finding.

R4. The intended Purview capability baseline (what *should* exist)

Description: A statement of **which Purview capabilities the organization intends to run** — e.g. "DLP on email + endpoints, sensitivity labels tenant-wide, a 7-year retention policy on finance sites, unified audit log on, insider risk for departing employees" — so a **missing** policy can be told apart from a **deliberately-not-adopted** capability. **Why:** A Gap is only a Gap against intent. A tenant with no insider-risk policy is **healthy** if the organization never intended insider risk, and a **Gap** if it did. The check needs the intended baseline to score fairly. **Verify:** The baseline is written down (even briefly) and confirmed with the compliance owner. **Implement if missing:** Agree the intended baseline with the compliance/records/security owners. Where a capability is **unlicensed**, note it as a **licensing decision**, not a drift Gap.

R5. Role-review boundaries and privacy posture for sensitive surfaces

Description: Agreement on **how** the role review and any insider-risk read are conducted: least-privilege, no snooping into case contents, and — for insider risk / communication compliance — respecting **role separation** and **privacy-by-design** (report presence and privacy posture, never case detail or any person's identity beyond what a role is authorized to see). **Why:** Insider Risk and Communication Compliance are **HR/privacy-sensitive**. A health check must report *that* the capability is configured and its **privacy guardrails are on** — not *what* it found about anyone. The role review itself touches who-can-see-what and must be handled carefully. **Verify:** Confirm with HR/privacy that the check reports **posture only** on sensitive surfaces, and that the role reviewer is authorized. `OPSHEALTH-Discovery.ps1`/`OPSHEALTH-ScopeCheck.ps1` report IRM **presence + privacy posture** only. **Implement if missing:** Agree the boundary in writing before assessing insider-risk or communication -compliance surfaces. When in doubt, report **posture**, not content.

R6. A named owner for each domain (so findings become actions)

Description: A named **owner** for each remediation domain — typically **DLP**, **Information Protection (labels)**, **Data Lifecycle / Records**, **Audit**, **Insider Risk**, and **Identity / Roles** — so every entry in the Remediation Register has someone accountable to close it. **Why:** A health check that produces findings nobody owns changes nothing. Severity + owner is what turns the register into a roadmap. (Aligns with category GR §5.) **Verify:** Each domain in the register maps to a named owner; the compliance/records/privacy/legal determinations sit with the correct governance function, not with IT. **Implement if missing:** Assign owners before you circulate the register; unowned High-severity findings are the ones that rot.

R7. Change control for the single safe direct fix (audit-log re-enable) and for downstream remediation

Description: A **change record** covering (a) the one direct fix this solution may commit — **re-enabling the unified audit log** — and (b) the downstream remediations that the owning capability solutions will perform (which they pilot in **simulation/test/report-only** under their own change control). Re-enabling the unified audit log is a **write** action; it needs an approver even though it is safe and reversible. **Why:** Even a safe

re-enable is a configuration change in a production tenant; and the higher-impact remediations (DLP/labels/retention/insider-risk) are high-impact and must be piloted, never flipped on from a health check. **Verify:** A change ticket names the approver and the rollback for the audit-log re-enable; `OPSHEALTH-Remediate.ps1` runs `-WhatIf` (plan only) until you pass `-Commit`. **Implement if missing:** Record the change; run the remediation planner `-WhatIf` first; commit the audit-log re-enable only after sign-off; route all other fixes to their owning solution.

R8. Compliance, records, privacy & legal stakeholders (determinations stay with them)

Description: Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy — so that any finding touching a **retention schedule, records file plan, legal hold, regulatory obligation, or insider-risk/ communication-compliance policy** is routed to them for the **determination**, not decided by the assessor. **Why:** The health check provides the **technical** state and evidence; **whether** a retention schedule is right, a hold is required, or a regulation applies is a **legal/records/compliance determination** (category GR §5). Several surfaces are HR/legal-sensitive. **Verify:** Confirm the applicable policies, the records schedule owner, and named approvers before the register's regulatory/records/legal items are actioned. **Implement if missing:** Identify the owners and capture the requirements; mark such register entries **"determination required"** and route them accordingly.

Checklist

- Category GR §1–§6 verified (licensing, roles, toolchain, data scope, stakeholders, pilot/change control)
- **Read** access to the in-scope surfaces (DLP, labels, retention, audit, roles) — `OPSHEALTH-Discovery.ps1`
- Modules installed (`ExchangeOnlineManagement`, `Microsoft.Graph`); modern auth works on all three connects
- **In-scope workload + data map** captured (the Coverage yardstick) — R3
- **Intended capability baseline** written down (missing vs not-adopted vs unlicensed) — R4
- Role-review + insider-risk **privacy boundary** agreed (posture only, role separation) — R5
- A **named owner** per domain (DLP / Labels / Lifecycle / Audit / Insider Risk / Roles) — R6
- Change control for the **audit-log re-enable** and downstream remediation — R7
- Compliance/records/privacy/legal stakeholders engaged; determinations routed to them — R8

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

Vendor boundary (applies to every Purview solution). *The* ***Microsoft Purview portal*** *(purview.microsoft.com), the* ***Microsoft 365 admin center****,* ***Security & Compliance PowerShell*** *(`Connect-IPPSSession`), and* ***Microsoft Graph*** *are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily* ***licensing-/plan-dependent*** *(many require* ***Microsoft 365 E5*** *or the* ***E5 Compliance*** *add-on; some are in preview) and the portal is* ***actively evolving*** *— capability availability and portal paths are* ***cited or flagged "verify your licensing/tenant"****, never asserted or invented.* ***Regulatory, records-retention, and legal determinations*** *(retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are* ***deferred to the organization's compliance, records, privacy, and legal functions*** *— the solutions provide the technical configuration method and evidence, not the determination. Automation is* ***PowerShell*** *(Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

• **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

• **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

• **Why required:** Purview policies key off the in-scope workloads and the classification scheme.

• **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.

• **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

Implementation — Microsoft Purview Health Check

This is the ordered, do-this-then-that guide for running the health check, scoring it, building the remediation register, and closing the gaps. It follows the standard model (**Assess** → **Design** → **Configure** → **Pilot (simulation/test)** → **Validate** → **Expand**), adapted for a **read-first assessment**: the "Assess" and "Design" phases are the health check itself; "Configure / Pilot" is remediation (routed to the owning capability, piloted in simulation/test — this solution commits only the one safe fix); "Validate/Expand" is re-scoring and turning the check into a recurring cadence. Every domain, rating, and register field is named verbatim so the scripts, testing, and troubleshooting line up. Replace `contoso.com` and sample names with your own.

> ***Portal note.*** *The Purview objects assessed here live in the* ***Microsoft Purview portal*** *(`purview.microsoft.com` → **Data Loss Prevention**, **Information Protection**, **Data Lifecycle Management**, **Audit**, **Insider Risk Management**, **Roles & scopes**). PowerShell (S&C PowerShell / Exchange Online / Graph) is used to* ***read*** *that state and to* ***plan*** *remediation. Where a capability is portal-only or its read surface differs by tenant/licence, the scripts say so —* ***verify against your licensing/tenant.***
>
> ***Read-only by default.*** *Phases 0–2 (Assess, Design) and Phase 4 (Validate) change* ***nothing****. The only write in this entire solution is the optional* ***unified-audit-log re-enable*** *in Phase 3, gated behind `-Commit`. All higher-impact remediation is handed to the owning capability solution, which pilots it in* ***simulation/test/report-only*** *first.*

Phase 0 — Assess (run the health check, read-only)

1. **Confirm read access and the toolchain (R1, R2).** Verify you can read DLP, labels, retention, audit, and role state in the portal, and that `Connect-IPPSSession`, `Connect-ExchangeOnline`, and `Connect-MgGraph` all succeed with an MFA-capable admin.

1. **Capture the yardsticks (R3, R4).** Have the **in-scope workload + data map** and the **intended capability baseline** to hand — the Coverage and Policy domains score against these.

1. **Run the inventory.** Run `OPSHEALTH-Discovery.ps1` (read-only). It inventories:

• **DLP** policies and their **Mode** (e.g. `Enable`, `TestWithNotifications`, `TestWithout...`, `Disable`) and enabled/health state.

• **Sensitivity labels** and **label policies** (published vs orphaned labels).

• **Retention / data-lifecycle** policies and **retention labels** (present, enabled, workloads).

• **Unified audit log** status (on/off) — via Exchange Online.

• **Insider-risk** policy **presence** and privacy posture (posture only — R5).

• **Compliance role groups** and their membership.

• **Gate 0:** the inventory completes and you can read each in-scope surface → continue. Where a surface cannot be read, record it as a **read-access** finding (not a drift Gap) and fix access.

1. **Run the coverage + role review.** Run `OPSHEALTH-ScopeCheck.ps1` (read-only) to map **which

workloads **each DLP/retention/label policy actually reaches, and to run the** least-privilege role review** (Global Admin count, single-member / empty role groups, membership size).

Phase 1 — Design (score the rubric, build the register)

Score every check **Healthy** → **At Risk** → **Gap** across the three domains, then record every non-Healthy result in the **Remediation Register**.

• **Policy health (P):**

• **Healthy** — the intended policy exists, is **enabled**, and is **healthy** (no error/unhealthy state).

• **At Risk** — the policy exists but is **stuck in simulation/test** past its pilot, partially disabled, or shows a health warning.

• **Gap** — an intended policy is **missing** or **disabled**, or the **unified audit log is off**.

• **Coverage (C):**

• **Healthy** — every in-scope workload has the intended DLP / retention / label coverage.

• **At Risk** — coverage exists but is incomplete (e.g. DLP on email only; labels published to a subset; endpoint actions intended but **no devices onboarded**).

• **Gap** — an in-scope workload has **no** coverage of an intended capability.

• **Roles (R):**

• **Healthy** — least privilege; more than one trained admin per critical role; no Global Admin used for routine compliance work; no stale/empty groups.

• **At Risk** — a single-member (SPOF) role group, mild over-membership, or a stale member.

• **Gap** — **Global Administrator sprawl**, an empty critical role group, or broad over-privilege.

Remediation Register — one row per non-Healthy finding, with these verbatim fields:

Field	Meaning
Finding	The drift/gap, in one line (e.g. "DLP policy `DLP-Finance` stuck in test 6 months").
Domain	**P** (Policy), **C** (Coverage), or **R** (Roles).
Rating	**At Risk** or **Gap**.
Severity	**High / Medium / Low** — audit-log off, missing DLP on a regulated workload, or GA sprawl are High.
Owner	The named domain owner (R6).
Owning capability	The solution that closes it (DLP / Labels / Lifecycle / Audit / Insider Risk / Roles), or "this solution" for the audit-log re-enable.

Field	Meaning
Recommended action	The next step (pilot in simulation/test; onboard devices; remove stale member; **determination required** for records/legal).

• **Determination items.** Any finding touching a **retention schedule, records file plan, legal hold,

regulatory obligation, or insider-risk/communication-compliance policy **is marked** "determination required"** and routed to compliance/records/privacy/legal (R8) — the register records the technical state, not the determination.

• **Licensing items.** A capability that is **absent because unlicensed** is a **licensing decision**,

not a drift Gap — tag it accordingly and **verify against your licensing/tenant.**

Phase 2 — Produce the scorecard (leadership rollup)

Run `OPSHEALTH-Report.ps1` (read-only) to roll the scored checks into the **health scorecard**: per-domain **Healthy / At Risk / Gap** counts, the overall **health rating**, and a CSV of the Remediation Register for circulation. This is the artifact leadership signs off on and the baseline you compare future runs against.

• **Pass/fail:** the scorecard reconciles with what the portal shows (the portal is authoritative); the

register has an owner and severity on every row.

Phase 3 — Remediate (plan first; commit only the one safe fix)

1. **Generate the remediation plan.** Run `OPSHEALTH-Remediate.ps1 -WhatIf` (the default). It prints

the prioritized plan — each register item, its severity, its owning capability, and the recommended action — and changes **nothing**.

1. **Commit the one safe, reversible fix (optional).** If the unified audit log is **off** and R7

change control is signed off, run `OPSHEALTH-Remediate.ps1 -EnableUnifiedAuditLog -Commit`. This is the **only** direct write in the solution (`Set-AdminAuditLogConfig -UnifiedAuditLogIngestionEnabled $true` via Exchange Online) — safe, documented, and reversible.

• **Pass/fail:** the unified audit log reads **enabled** after propagation. **Rollback:** re-run with

the disable path (or set it back in Exchange Online); audit ingestion can take time to reflect — **verify in your tenant.**

1. **Route every other remediation to its owning capability solution.** DLP, sensitivity labels,

retention/records, and insider-risk changes are **high-impact** — they are **not** flipped on from this health check. Each is handed to its owning solution and **piloted in simulation/test/report-only** first, validated, then enforced under that solution's discipline. The register's **Owning capability** field says where each one goes.

1. **Determination items go to governance, not to IT.** Records/legal/regulatory items wait on the

relevant function's determination (R8) before any configuration.

> ***Hard rule.*** *Do* ***not*** *use this solution to enable/enforce DLP, publish or change labels, create or change retention/records policies, or configure insider risk. Those belong to the owning capability solutions, piloted first. This solution* ***finds, prioritizes, and (only) re-enables the audit log.***

Phase 4 — Validate (re-score, read-only)

Work the gates in `05-testing.md`. In summary:

- Re-run `OPSHEALTH-Validate.ps1` (read-only) and confirm each remediated item now reads **Healthy**

against its criterion.

- Confirm the **unified audit log** is on (if remediated), that no intended DLP/retention/label policy

is missing or disabled, that coverage matches the in-scope map, and that the role review is clean.

- Re-run `OPSHEALTH-Report.ps1` and confirm the overall **health rating** improved and the register's

closed items are gone.

Phase 5 — Expand (make it recurring)

1. **Set a cadence.** Re-run the read-only health check on a schedule (e.g. quarterly, and before any

audit) so drift is caught early — the scripts are re-runnable and produce a comparable scorecard.

1. **Track the trend.** Keep each run's scorecard; a falling count of At Risk/Gap items over time is the

evidence that the estate is staying healthy.

1. **Keep determinations with governance.** As the estate matures, records/retention/legal items keep

flowing to their owners; the health check keeps surfacing the **technical** state, not the decision.

Rollback summary

Change	Rollback
Unified audit log re-enabled and you must revert	Disable ingestion again in Exchange Online (`Set-AdminAuditLogConfig -UnifiedAuditLogIngestionEnabled $false`); propagation is not instant — verify in tenant.
A downstream remediation (DLP/label/retention/IRM) caused disruption	It was piloted in **simulation/test/report-only** by its owning solution — revert per that solution's rollback (set mode back to test/disable), not here.
Register mis-scored a licensing gap as a drift Gap	Re-tag as a **licensing decision**; re-run `OPSHEALTH-Report.ps1` to correct the scorecard.
A determination item was actioned before governance signed off	Halt; route to records/legal/compliance; the health check only surfaces technical state (R8).

> ***The health check changes almost nothing by design.*** *Its value is the* ***evidence and the ordered register*** *— the fixes happen, safely and piloted, in the capability solutions that own them.*

Testing & Validation — Microsoft Purview Health Check

Run these gates in order. Each has an explicit **pass/fail** signal. Because this is a **read-only assessment**, the gates confirm that the health check **read the estate completely and scored it correctly** — and, after remediation, that each fixed item now reads **Healthy**. The **portal is authoritative**; where a script cannot read a surface, that is a finding about **access**, not a verdict about the estate.

Use the scripts where noted; they are read-only or `-WhatIf` and change nothing you don't explicitly `-Commit`.

Gate 1 — Read access and toolchain are in place

Why: The check can only score what it can read; missing read access produces false Gaps. **How:** Confirm `Connect-IPPSSession`, `Connect-ExchangeOnline`, and `Connect-MgGraph` all succeed with an MFA-capable admin. Run `OPSHEALTH-Discovery.ps1` and confirm it reaches DLP, labels, retention, audit, and role surfaces. **Pass:** All three connections succeed; the inventory reads every in-scope surface without an access error. **Fail:** A surface returns access-denied → this is a **read-access** finding; fix access (R1) and re-run before scoring that domain.

Gate 2 — The inventory is complete (Discovery)

Why: A partial inventory under-reports drift. **How:** Run `OPSHEALTH-Discovery.ps1`. Confirm it returns DLP policies (with **Mode**), sensitivity labels + label policies, retention/lifecycle policies + labels, unified-audit-log status, insider-risk **presence/posture**, and role-group membership. **Pass:** Every in-scope surface is inventoried; counts reconcile with a spot-check in the portal. **Fail:** A surface is empty when the portal shows objects → confirm read role and cmdlet availability (**verify in tenant**) before trusting the score.

Gate 3 — Policy health (P) is scored against intent

Why: A policy can exist but be **disabled** or **stuck in test** — both are drift. **How:** For each intended policy (R4 baseline), confirm its **Mode**/enabled/health state. `OPSHEALTH-ScopeCheck.ps1` / `OPSHEALTH-Validate.ps1` report mode and health. **Pass:** Each intended DLP/retention/label policy is **enabled and healthy** → **Healthy**; anything **stuck in simulation/test** past pilot or showing a health warning is **At Risk**; anything **missing or disabled**, or the **audit log off**, is a **Gap** — and every non-Healthy item is in the register. **Fail:** A disabled/stuck/missing policy is not reflected in the score/register → re-score.

Gate 4 — The unified audit log status is correctly determined

Why: The audit log is the defensibility evidence base; off = High-severity Gap. **How:** `OPSHEALTH-Discovery.ps1` / `OPSHEALTH-Validate.ps1` report unified-audit-log status via Exchange Online. **Pass:** Status is read correctly (**on** = Healthy; **off** = a **High**-severity Gap in the register). **Fail:** Status cannot be read, or is misreported → confirm Exchange Online connectivity and the `Get-AdminAuditLogConfig`/organization-config read; **verify in tenant**.

Gate 5 — Coverage (C) is scored against the in-scope map

Why: "We have DLP" often means "on email only" — the blind spots are the risk. **How:** `OPSHEALTH-ScopeCheck.ps1` maps each policy's **workloads** (Exchange / SharePoint / OneDrive / Teams / endpoints) against the R3 in-scope map. **Pass:** Each in-scope workload's intended coverage is present → **Healthy**; partial coverage (e.g. labels to a subset, endpoint intended but **no devices onboarded**) → **At Risk**; an in-scope workload with **no** coverage of an intended capability → **Gap** — all recorded. **Fail:** A workload blind spot is not surfaced → re-check the policy's location/scope settings and the in-scope map.

Gate 6 — Roles (R) least-privilege review is clean or flagged

Why: Role drift (GA sprawl, SPOF, stale/empty groups) is a real, auditable risk. **How:** `OPSHEALTH-ScopeCheck.ps1` reports compliance role-group membership, Global Admin count, single-member and empty role groups. **Pass:** Least privilege confirmed → **Healthy**; a single-member (SPOF) group, mild over-membership, or a stale member → **At Risk**; **Global Admin sprawl**, an empty **critical** role group, or broad over-privilege → **Gap** — all recorded with owner **Identity / Roles**. **Fail:** An obvious over-privilege (e.g. many Global Admins) is not flagged → re-run and confirm the directory-role read (Graph) succeeded.

Gate 7 — Insider-risk / sensitive surfaces reported as POSTURE only

Why: Insider Risk and Communication Compliance are HR/privacy-sensitive. **How:** Review the Discovery output for those surfaces. **Pass:** The check reports **presence + privacy posture** (e.g. anonymization on, role separation intact) **only** — no case contents, no person's identity, no judgement. **Fail:** Any case detail or identity beyond authorized role scope appears → **stop**; the review breached the privacy boundary (R5). Report posture only.

Gate 8 — The scorecard and register are complete and owned

Why: Findings without severity and an owner do not get fixed. **How:** Run `OPSHEALTH-Report.ps1`; inspect the scorecard + Remediation Register CSV. **Pass:** Every non-Healthy check is a register row with **Domain**, **Rating**, **Severity**, **Owner**, **Owning capability**, and **Recommended action**; the overall **health rating** reconciles with the per-domain counts and the portal. **Fail:** A finding lacks an owner/severity, or the rollup diverges from the portal → the portal is authoritative; reconcile and re-issue.

Gate 9 — Remediation planned `-WhatIf`; only the audit-log fix committed

Why: The health check must not enable/enforce high-impact capabilities. **How:** Run `OPSHEALTH-Remediate.ps1 -WhatIf`; if committing the audit-log fix, run with `-EnableUnifiedAuditLog -Commit` under R7 change control. **Pass:** `-WhatIf` prints the plan and changes nothing; the only committed change is the **unified audit-log re-enable**; all DLP/label/retention/insider-risk items are routed to their owning capability (piloted in simulation/test there), not enabled here. **Fail:** Any attempt to enable/enforce DLP/labels/retention/insider risk **from this solution** → stop; that belongs to the owning solution, piloted first.

Gate 10 — Re-score confirms improvement (post-remediation)

Why: Closing the loop proves the remediation worked and the estate is healthier. **How:** After remediation, re-run `OPSHEALTH-Validate.ps1` and `OPSHEALTH-Report.ps1`. **Pass:** Each remediated item now reads **Healthy**; the overall **health rating** improved; closed items are gone from the register; determination items remain pending their governance owner (expected). **Fail:** A remediated item still reads non-Healthy → confirm the fix landed in the owning capability and propagated (**verify in tenant**), then re-score.

Exit criteria (all must be true to call the health check complete)

- Read access + toolchain verified (Gate 1)
- Inventory complete and reconciled with the portal (Gate 2)
- Policy health scored against intent (Gate 3)
- Unified audit-log status correctly determined (Gate 4)
- Coverage scored against the in-scope map (Gate 5)
- Role least-privilege review clean or flagged (Gate 6)
- Insider-risk / sensitive surfaces reported as posture only (Gate 7)
- Scorecard + Remediation Register complete and owned (Gate 8)
- Remediation planned `-WhatIf`; only the audit-log fix committed (Gate 9)
- Re-score confirms improvement; determinations routed to governance (Gate 10)

Troubleshooting — Microsoft Purview Health Check

The common failure modes when running a read-only Purview health check, and how to resolve them. Because this solution **reads and scores** (its only write is the gated audit-log re-enable), almost every issue is a **read-access**, **cmdlet-availability**, or **scoring-interpretation** problem — not a change that broke something. The **portal is authoritative**; when a script and the portal disagree, trust the portal and fix the read. Objects, domains (**P/C/R**), and the register fields are named verbatim from the Implementation Guide.

A surface returns access-denied → a false "Gap"

- **Symptom:** `OPSHEALTH-Discovery.ps1` / `OPSHEALTH-ScopeCheck.ps1` can't read DLP, labels, retention,

audit, or roles, and the domain looks empty.

- **Likely cause:** The account lacks the **read** role for that surface (R1) — an empty read is not the

same as an empty estate.

- **Resolve:** Record it as a **read-access** finding, **not** a drift Gap; assign the least-privilege

read role (e.g. Compliance/Global Reader, the relevant View-Only role group) under change control; re-run before scoring that domain. Do **not** over-grant write access "to be safe."

A cmdlet is not available in the tenant

• **Symptom:** `Get-InsiderRiskPolicy`, `Get-Label`, or another cmdlet is missing after a successful connect.

• **Likely cause:** The capability is **portal-managed** or **plan-gated** (E5 / E5 Compliance; some preview), or the module is old.

• **Resolve:** Update the module (`Install-Module ExchangeOnlineManagement`, `Install-Module Microsoft.Graph`); where the cmdlet genuinely doesn't exist, confirm the surface in the **portal** (Insider Risk is largely portal-managed — report **presence + privacy posture** only) and **verify against your licensing/tenant.** A plan-gated absence is a **licensing** note, not a drift Gap.

The unified audit-log status is ambiguous or unreadable

• **Symptom:** `Get-AdminAuditLogConfig` errors, or the on/off status looks wrong.

• **Likely cause:** Not connected to **Exchange Online** (separate from S&C PowerShell), missing read access, or ingestion still propagating after a change.

• **Resolve:** `Connect-ExchangeOnline` and confirm `Get-AdminAuditLogConfig` read access; re-check `UnifiedAuditLogIngestionEnabled`. After re-enabling with `OPSHEALTH-Remediate.ps1 -EnableUnifiedAuditLog -Commit`, allow propagation — status is not instant; **verify in tenant.**

Coverage is mis-mapped (a policy looks like it covers less/more than it does)

• **Symptom:** `OPSHEALTH-ScopeCheck.ps1` reports a workload blind spot that the portal contradicts, or the `Workload` field is blank.

• **Likely cause:** The policy's location/scope is set in a way the cmdlet's `Workload` field doesn't fully express, or coverage was scored **without** the R3 in-scope map.

• **Resolve:** Open the policy in the portal and read its **locations** directly (authoritative); score Coverage against the **R3 in-scope workload map** — an intended-but-uncovered workload is a Gap, a not-intended one is not. "We have DLP" often means "email only" — confirm each workload.

Role read is incomplete (Global Admin count missing)

• **Symptom:** The role review shows compliance role groups but no Global Administrator count.

• **Likely cause:** The **Microsoft Graph** directory-role read didn't run — module not imported, not connected, or missing `RoleManagement.Read.Directory` / `Directory.Read.All`.

• **Resolve:** Import `Microsoft.Graph.Identity.DirectoryManagement`, `Connect-MgGraph` with the

directory-read scopes, and re-run `OPSHEALTH-ScopeCheck.ps1`. GA sprawl (over the threshold), empty critical role groups, and single-member (SPOF) groups are role **Gaps/At Risk** to record with owner **Identity / Roles**.

Licensing gap scored as a drift Gap (confusion)

- **Symptom:** A capability that was never adopted (or isn't licensed) is scored as a **Gap**.
- **Likely cause:** Missing the **R4 intended capability baseline** — a Gap is only a Gap **against intent**.
- **Resolve:** Compare against the R4 baseline: **missing-but-intended** = Gap; **not-adopted** = Healthy; **unlicensed** = a **licensing decision**, tagged as such and **verify against your licensing/tenant** — not a drift Gap. Re-tag and re-run `OPSHEALTH-Report.ps1` to correct the scorecard.

"Stuck in test" flagged on a fresh pilot (false At Risk)

- **Symptom:** A DLP policy in `Test*` mode is flagged **At Risk**, but it's a legitimate current pilot.
- **Likely cause:** The heuristic can't tell a fresh pilot from long-term drift.
- **Resolve:** Confirm the pilot start date; a policy in simulation/test **past its pilot window** is At Risk (drift), a **current** pilot is expected. Annotate the register accordingly.

Insider-risk / sensitive surface — how to stay on the right side of the privacy line

- **Symptom:** Unsure how much to report on Insider Risk / Communication Compliance.
- **Likely cause:** These are **HR/privacy-sensitive** (R5).
- **Resolve:** Report **presence + privacy posture only** (e.g. anonymization on, role separation intact) — **never** case contents or any person's identity. If any case detail appears beyond your authorized role scope, **stop**: the review breached the boundary. Posture, not content.

The scorecard diverges from the portal

- **Symptom:** `OPSHEALTH-Report.ps1` rollup doesn't match what the portal shows.
- **Likely cause:** A read gap, stale CSV, or a mis-scored row.
- **Resolve:** The **portal is authoritative** — reconcile the register to it, fix any read-access finding, ensure every non-Healthy row has an **owner** and **severity**, and re-issue.

A determination item was actioned before governance signed off

- **Symptom:** A retention-schedule / records / legal-hold / regulatory item was changed by IT.
- **Likely cause:** A **"determination required"** item was treated as a technical fix.
- **Resolve: Halt**; route it to compliance/records/privacy/legal (R8). The health check surfaces

technical state, not the determination — those decisions are theirs.

Key reminders

Rule	Why
Empty read ≠ empty estate	An unreadable surface is a **read-access** finding, not a Gap
The portal is authoritative	When a script disagrees, trust the portal and fix the read
Score against **intent** (R4)	Missing vs not-adopted vs unlicensed are different verdicts
Posture only on IRM/CC	HR/privacy-sensitive — presence + guardrails, never content
One direct write only	The gated **audit-log re-enable**; everything else is planned/routed
Determinations to governance	Records/retention/legal/regulatory decisions are not IT's

Best Practices — Microsoft Purview Health Check

Field-tested guidance for running a Purview health check that produces **evidence and action**, not just a wall of red. It follows the same order the check runs: read everything, score against **intent**, prioritize by **severity and owner**, and hand fixes to the solutions that own them — piloted, never flipped.

1. Score against intent, not against a maximum

• A "Gap" is only a Gap against what the organization **intended** to run (R4). A tenant with no insider-risk policy is **Healthy** if it never adopted insider risk, and a **Gap** if it did. Grade against the **baseline**, or you will manufacture false findings and lose credibility.

• Separate **"missing"** from **"deliberately not adopted"** from **"unlicensed"**. Only the first is a drift Gap; the third is a **licensing decision** — tag it and **verify against your licensing/tenant.**

2. Read-only first — the check itself changes nothing

• The whole assessment runs **read-only**. The single exception — re-enabling the unified audit log — is offered `-WhatIf` **first** and committed only with `-Commit` under change control. Keep it that way; a health check that starts *changing* the estate is no longer a health check.

• Never use the health check to enable/enforce DLP, publish labels, create retention/records policies, or configure insider risk. Those are **high-impact** and belong to the owning capability solution, **piloted in simulation/test/report-only** first.

3. "Stuck in test" is drift, not safety

• A DLP or retention policy left in **simulation/test** for months is not "being careful" — it is

not protecting anything while everyone assumes it is. Score long-lived test-mode policies as **At Risk** and put a date on graduating them (via the owning solution's pilot→enforce path).

• Equally, a **disabled** policy that everyone forgot to re-enable after an incident is a **Gap**, not a

historical artifact.

4. Coverage is where the real risk hides

• "We have DLP / labels / retention" almost always hides a **workload blind spot** — email but not

endpoints, a label published to a pilot group two years ago, retention on SharePoint but not OneDrive. **Map every policy to its workloads** (`OPSHEALTH-ScopeCheck.ps1`) and compare to the in-scope map — the gaps between *intended* and *actual* coverage are the findings that matter most.

• For endpoint actions, **no onboarded devices** means endpoint DLP does nothing regardless of policy —

check device onboarding, not just policy existence.

5. Treat the unified audit log as foundational

• The unified audit log is the **evidence base** for eDiscovery, insider risk, and every "prove what

happened" question. If it is **off**, that is a **High**-severity Gap and the one thing this solution will safely re-enable directly. Enable it early — audit data is not retroactive.

6. Least privilege is a health metric, not a one-time setup

• Review role groups every run: **Global Admin sprawl**, **single-member (SPOF)** groups, **empty**

critical groups, and **stale** members who moved on. All are auditable risks that accrue silently.

• Prefer **more than one** trained admin per critical compliance role (no single point of failure), and

no Global Admin for routine compliance work. Keep **Insider Risk** roles **separated** — do not fold them into general compliance admin for convenience.

7. Respect privacy on the sensitive surfaces

• On **Insider Risk** and **Communication Compliance**, report **posture only** — that the capability

exists and its **privacy guardrails (anonymization, role separation)** are on — **never** case contents or any person's identity. A health check must not become a snooping tool (R5).

• Conduct the role review as least-privilege and authorized; who-can-see-what is itself sensitive.

8. Severity + owner is what turns findings into fixes

• Every non-Healthy result gets a **severity** (High/Medium/Low) and a **named owner** (R6). Unowned

High findings are the ones that rot for another year. High = audit log off, missing DLP on a regulated workload, GA sprawl; Low = cosmetic naming, a single stale reviewer.

• Sequence remediation **High** → **Medium** → **Low**, and route each to its **owning capability** so the fix happens where the expertise and the pilot discipline live.

9. Defer determinations — you provide the method, not the verdict

• Any finding touching a **retention schedule, records file plan, legal hold, regulatory obligation, or insider-risk/communication-compliance policy **is marked** "determination required" **and routed to** compliance/records/privacy/legal **(R8). The health check reports the** technical **state and evidence —** whether** it is correct is their call, not the assessor's.

10. Make it recurring — drift comes back

• A one-time health check ages the moment it is delivered. Re-run it on a **cadence** (quarterly, and before any audit) and keep each scorecard. A **falling** trend of At Risk/Gap items over time is the proof the estate is staying healthy — and the recurring baseline makes the *next* drift obvious fast.

11. Verify against your tenant — the surfaces evolve

• Purview capabilities are **licensing-dependent** and the portal **changes**. Cmdlet availability, policy modes, and read surfaces differ by tenant and plan. Where a script notes a capability may be portal-only or plan-gated, **confirm against your licensing/tenant** rather than assuming — and treat preview capabilities as preview.

Deliverables Summary — Microsoft Purview Health Check

Everything included in this solution, and what each piece is for.

Documentation

Document	Purpose
Solution Overview (`01-solution-overview.md`)	What the health check delivers, the three domains (Policy / Coverage / Roles), the Healthy→At Risk→Gap rubric, the assessment model, and the read-only-plus-one-safe-fix boundary.
Requirements (`02-requirements.md`)	Health-check-specific prerequisites — read access to the in-scope surfaces, the S&C PowerShell / Exchange Online / Graph toolchain, the in-scope workload map and intended capability baseline (the yardsticks), the role-review/insider-risk privacy boundary, domain owners, change control for the audit-log fix, and stakeholder routing for determinations.

Document	Purpose
Implementation (`03-implementation.md`)	Ordered steps: run the read-only inventory + coverage/role review, score the rubric, build the Remediation Register, produce the scorecard, plan remediation `-WhatIf` (commit only the audit-log re-enable), route everything else to its owning capability, re-score, and make it recurring — with pass/fail gates and rollback.
Testing & Validation (`05-testing.md`)	Ten pass/fail gates from read access and inventory completeness through per-domain scoring, the audit-log determination, the privacy boundary, register completeness, and the post-remediation re-score.
Troubleshooting (`06-troubleshooting.md`)	The common health-check failure modes (surface unreadable / false Gap, cmdlet not available, audit-log status ambiguous, coverage mis-mapped, role read incomplete, licensing vs drift confusion) and how to resolve them.
Best Practices (`07-best-practices.md`)	Score against intent, read-only first, "stuck in test" is drift, coverage is where risk hides, audit log is foundational, least privilege as a metric, respect privacy, severity+owner, defer determinations, make it recurring, verify against tenant.
Executive Summary (`executive-summary.md`)	One-page business framing for decision-makers.

PowerShell tools (5)

All tools are **read-only or `-WhatIf` by default**, authenticate **interactively** (no credentials in code), and use **Security & Compliance PowerShell** (`Connect-IPPSSession`), **Exchange Online** (`Connect-ExchangeOnline`), and/or **Microsoft Graph** (`Connect-MgGraph`). The health check **reads and scores**; the one direct write (re-enabling the unified audit log) is gated behind `-Commit`.

Script	Type	What it does
`OPSHEALTH-Discovery.ps1`	Read-only	**Estate inventory**: enumerates DLP policies (with **Mode**), sensitivity labels + label policies, retention/lifecycle policies + retention labels, unified-audit-log status (via Exchange Online), insider-risk policy **presence + privacy posture** (posture only), and compliance role groups. Changes nothing — the raw material for scoring.

Script	Type	What it does
`OPSHEALTH-ScopeCheck.ps1`	Read-only	**Coverage + least-privilege role review** (the scope/health helper): maps each DLP/retention/label policy to the **workloads** it reaches (Exchange / SharePoint / OneDrive / Teams / endpoints) against the in-scope map, and reviews role groups for **Global Admin sprawl, single-member (SPOF), empty groups, and over-membership**.
`OPSHEALTH-Validate.ps1`	Read-only	**Validation checker**: reports each check as a pass/fail line aligned to `05-testing.md` — policy enabled/healthy/not-stuck, audit log on, coverage matches intent, role review clean — for scoring and for the post-remediation re-score.
`OPSHEALTH-Remediate.ps1`	`-WhatIf` default	**Remediation-plan generator**: prints the prioritized plan (finding → severity → owning capability → recommended action) and changes nothing by default. Its **only** commit path, `-EnableUnifiedAuditLog -Commit`, performs the single safe, reversible fix — re-enabling the unified audit log; all other fixes are routed to their owning capability solution to be piloted there.
`OPSHEALTH-Report.ps1`	Read-only	**Health scorecard rollup**: summarizes per-domain **Healthy / At Risk / Gap** counts and the overall **health rating**, and exports the **Remediation Register** to CSV for a leadership rollup and as the baseline for the next run.

Named artifacts (verbatim — used across docs, scripts, and tests)

- **Domains: Policy health (P)**, **Coverage (C)**, **Roles (R)**
- **Rating scale: Healthy** → **At Risk** → **Gap**, rolling up to an overall **health rating**
- **Remediation Register** fields: **Finding · Domain · Rating · Severity (High/Medium/Low) · Owner · Owning capability · Recommended action**

• **The one direct fix:** re-enable the **unified audit log** (`-EnableUnifiedAuditLog -Commit`), safe and reversible; everything else is **planned** and routed to its owning capability

What this solution deliberately does NOT do

• It does **not** enable, enforce, publish, or change **DLP, sensitivity labels, retention/records, or insider-risk **configuration — those are high-impact and belong to the owning capability solution,** piloted in simulation/test/report-only** first. The health check finds and prioritizes; it does not flip.

• It does **not** make **records, retention-schedule, legal-hold, or regulatory determinations** — those are compliance/records/privacy/legal's, and such findings are marked **"determination required"**.

• It does **not** expose **insider-risk / communication-compliance case contents** or any person's identity — it reports **posture only** on those surfaces.

• It does **not** invent portal paths or cmdlets — where a surface is portal-only or plan-gated, the tools say so and defer to **"verify against your licensing/tenant."**

• Its **only** direct write is the reversible **unified-audit-log re-enable**, gated behind `-Commit`.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OPSHEALTH-Discovery.ps1

Purpose: Connects to Security & Compliance PowerShell (and Exchange Online for the unified-audit-log status) and inventories the Microsoft Purview estate so the HEALTH CHECK can score it. It reports DLP policies with their Mode, sensitivity labels and label policies, retention/data-lifecycle policies and retention labels, the unified-audit-log status, insider-risk policy PRESENCE and privacy posture (posture only — never case contents), and compliance role groups. Read-only — it changes nothing. This is the raw material the scoring in 03-implementation.md works from. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-Discovery.ps1 -CsvPath .\opshealth-inventory.csv

How to run: powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-Discovery.ps1 -CsvPath .\opshealth-inventory.csv

OPSHEALTH-Remediate.ps1

Purpose: Generates the remediation plan for the health check and changes NOTHING by default. It prints the prioritized plan (finding -> severity -> owning capability -> recommended action) from a Remediation-Register CSV (or a live re-scan), and routes DLP / label / retention / insider-risk fixes to their OWNING capability solution (piloted in simulation/test there). Its ONLY commit path, -EnableUnifiedAuditLog -Commit, performs the single safe, reversible direct fix — re-enabling the unified audit log via Exchange Online. It never enables/enforces DLP, labels, retention, or IRM. HOW TO RUN Plan only (default): powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-Remediate.ps1 -WhatIf From a register CSV: .\OPSHEALTH-Remediate.ps1 -RegisterCsv .\opshealth-register.csv -WhatIf Commit the one safe fix: .\OPSHEALTH-Remediate.ps1 -EnableUnifiedAuditLog -Commit Prereqs: ExchangeOnlineManagement; MFA-capable admin. -Commit needs R7 change-control sign-off. EXAMPLE .\OPSHEALTH-Remediate.ps1 -WhatIf EXPECTED RESULTS The prioritized plan (WhatIf) with changes = none; with -EnableUnifiedAuditLog -Commit the unified audit log reads enabled after propagation (the only direct write). All other items are routed to their owning capability solution — never flipped on here. .NOTES White-label. -WhatIf by default; only -Commit writes, and only the audit-log re-enable. No credentials in code. > [CmdletBinding()] param([string] $RegisterCsv, [switch] $EnableUnifiedAuditLog, [switch] $Commit, [switch] $WhatIf)

How to run: Plan only (default): powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-Remediate.ps1 -WhatIf From a register CSV: .\OPSHEALTH-Remediate.ps1 -RegisterCsv .\opshealth-register.csv -WhatIf Commit the one safe fix: .\OPSHEALTH-Remediate.ps1 -EnableUnifiedAuditLog -Commit Prereqs: ExchangeOnlineManagement; MFA-capable admin. -Commit needs R7 change-control sign-off. EXAMPLE .\OPSHEALTH-Remediate.ps1 -WhatIf EXPECTED RESULTS The prioritized plan (WhatIf) with changes = none; with -EnableUnifiedAuditLog -Commit the unified audit log reads enabled after propagation (the only direct write). All other items are routed to their owning capability solution — never flipped on here. .NOTES White-label. -WhatIf by default; only -Commit writes, and only the audit-log re-enable. No credentials in code. > [CmdletBinding()] param([string] $RegisterCsv, [switch] $EnableUnifiedAuditLog, [switch] $Commit, [switch] $WhatIf)

OPSHEALTH-Report.ps1

Purpose: Produces the leadership rollup for the health check: per-domain (Policy P / Coverage C / Roles R) Healthy / At Risk / Gap counts and an overall HEALTH RATING, and exports the Remediation Register to CSV as the baseline for the next run. It reads the scored register (a CSV you built from the Discovery + ScopeCheck findings). Read-only — it summarizes and exports, it changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-Report.ps1 -RegisterCsv .\opshealth-register.csv powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-Report.ps1 -RegisterCsv .\in.csv -OutCsv .\opshealth-scorecard.csv The register CSV columns: Finding, Domain (P/C/R), Rating (Healthy/At Risk/Gap), Severity, Owner, 'Owning capability', 'Recommended action'. EXPECTED RESULTS A per-domain Healthy/At Risk/Gap table, the overall health rating, and (with -OutCsv) the register written out for circulation. PASS = the rollup reconciles with the portal (the portal is authoritative) and every non-Healthy row has an owner and severity. .NOTES White-label. Read-only. No credentials in code. The portal is authoritative — reconcile before issuing. > [CmdletBinding()] param([Parameter(Mandatory=$true)][string] $RegisterCsv, [string] $OutCsv)

How to run: powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-Report.ps1 -RegisterCsv .\opshealth-register.csv powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-Report.ps1 -RegisterCsv .\in.csv -OutCsv .\opshealth-scorecard.csv The register CSV columns: Finding, Domain (P/C/R), Rating (Healthy/At Risk/Gap), Severity, Owner, 'Owning capability', 'Recommended action'. EXPECTED RESULTS A per-domain Healthy/At Risk/Gap table, the overall health rating, and (with -OutCsv) the register written out for circulation. PASS = the rollup reconciles with the portal (the portal is authoritative) and every non-Healthy row has an owner and severity. .NOTES White-label. Read-only. No credentials in code. The portal is authoritative — reconcile before issuing. > [CmdletBinding()] param([Parameter(Mandatory=$true)][string] $RegisterCsv, [string] $OutCsv)

OPSHEALTH-ScopeCheck.ps1

Purpose: The scope/health helper for the health check. Reads each DLP / retention / label policy and maps which WORKLOADS it actually reaches (Exchange / SharePoint / OneDrive / Teams / endpoints) so the Coverage (C) domain can be scored against the in-scope map (R3), and runs the least-privilege ROLE review (R) — compliance role-group membership plus, via Microsoft Graph, Global Admin count, single-member (SPOF) and empty role groups. Read-only — changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-ScopeCheck.ps1 powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-ScopeCheck.ps1 -CsvPath .\opshealth-coverage.csv

How to run: powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-ScopeCheck.ps1 powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-ScopeCheck.ps1 -CsvPath .\opshealth-coverage.csv

OPSHEALTH-Validate.ps1

Purpose: Emits one PASS/FAIL line per health-check criterion so the estate can be scored and, after remediation, re-scored. It checks: at least one intended DLP/retention/label policy is enabled and not stuck in test; the unified audit log is on; and the compliance role review is clean (no empty/SPOF role groups). Read-only — changes nothing. Use it for the initial score and for the Gate 10 post-remediation re-score. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-Validate.ps1 Prereqs: ExchangeOnlineManagement (Connect-IPPSSession + Connect-ExchangeOnline), MFA-capable read admin. Verify capability availability against your licensing/tenant. EXPECTED RESULTS A list of PASS/FAIL lines and an overall pass count. PASS overall = no intended policy missing/ disabled/stuck, audit log on, role review clean. Any FAIL is a Remediation-Register row. Exit code 0 when all pass, 1 when any fail (for pipeline use). .NOTES White-label. Read-only. Security & Compliance PowerShell / Exchange

Online. No credentials in code. > [CmdletBinding()] param()

How to run: powershell -ExecutionPolicy Bypass -File .\OPSHEALTH-Validate.ps1 Prereqs: ExchangeOnlineManagement (Connect-IPPSSession + Connect-ExchangeOnline), MFA-capable read admin. Verify capability availability against your licensing/tenant. EXPECTED RESULTS A list of PASS/FAIL lines and an overall pass count. PASS overall = no intended policy missing/ disabled/stuck, audit log on, role review clean. Any FAIL is a Remediation-Register row. Exit code 0 when all pass, 1 when any fail (for pipeline use). .NOTES White-label. Read-only. Security & Compliance PowerShell / Exchange Online. No credentials in code. > [CmdletBinding()] param()